Guide des vrais commits Git

01 But et règle d'authenticité

Objectif

Ce guide permet aux quatre membres du groupe 10 de produire un historique Git vérifiable. Un commit doit correspondre à une modification réellement comprise, réalisée et testée par la personne dont le nom apparaît comme auteur.

Ce qui prouve une contribution

Le nom et le courriel enregistrés dans le commit, le contenu précis de la modification, le message expliquant pourquoi elle a été faite et, si un dépôt distant est utilisé, le compte qui a envoyé la branche constituent ensemble la preuve la plus claire.

Interdictions

Ne pas partager une même identité Git entre tous les membres. Ne pas utiliser --author pour attribuer après coup le travail à quelqu'un d'autre. Ne pas modifier la date, ne pas réécrire l'historique pour fabriquer une participation et ne pas découper artificiellement des fichiers déjà terminés.

Avant de commencer

Créer une copie de sauvegarde de l'archive finale. Chaque membre choisit ensuite une amélioration réelle et limitée : correction de documentation technique, ajout d'un contrôle, amélioration d'un test ou clarification d'une procédure. La modification doit rester cohérente avec le périmètre du projet.

02 Méthode recommandée un clone par membre

Organisation

La méthode la plus claire consiste à donner à chaque membre son propre clone du dépôt, sur son compte utilisateur ou dans un dossier distinct. Le responsable du dépôt ajoute les autres membres comme collaborateurs si un dépôt GitHub ou GitLab est utilisé. Chaque membre travaille sur une branche portant son nom ou son sujet.

Configurer son identité

Dans son propre clone, le membre exécute : git config --local user.name "Prénom Nom" puis git config --local user.email "adresse vérifiée du compte". Il contrôle avec git config --local --get user.name et git config --local --get user.email. L'option --local évite de modifier les autres projets de l'ordinateur.

Créer une branche

Exécuter git switch -c contribution-prenom-sujet. Vérifier git status. Modifier uniquement les fichiers nécessaires, puis lancer les tests adaptés. Relire les différences avec git diff avant de préparer le commit.

Préparer précisément

Ajouter seulement les fichiers concernés avec git add chemin/du/fichier. Contrôler ensuite git diff --cached. Si un fichier a été ajouté par erreur, utiliser git restore --staged chemin/du/fichier. Aucun secret, .env, .runtime ou dossier .vagrant ne doit être préparé.

03 Créer vérifier et partager le commit

Message

Utiliser un sujet court comme docs(validation) préciser la preuve v4 ou test(network) vérifier les règles postgres. Ajouter un corps de cinq à quinze lignes qui explique le problème, le changement, le test effectué et la limite éventuelle. Éviter les messages vagues comme modification ou travail fini.

Commande

Créer le commit avec git commit. L'éditeur permet d'écrire le sujet, une ligne vide puis le corps. Contrôler ensuite git show --stat --oneline HEAD et git show HEAD pour vérifier l'auteur, le message et le contenu exact.

Partager

Si un dépôt distant existe, publier la branche avec git push -u origin contribution-prenom-sujet depuis le compte réel du membre. Ouvrir ensuite une demande de fusion ou faire relire le commit par le groupe. La fusion doit conserver l'auteur d'origine.

Même ordinateur

Si tous les membres utilisent le même ordinateur, employer quatre clones séparés est préférable. À défaut, chaque membre doit configurer l'identité locale juste avant son propre travail, la vérifier, créer son commit, puis se déconnecter du compte distant avant de laisser la place au suivant.

04 Plan pour les quatre membres et contrôle final

Exemples de contributions

Membre réseau : améliorer et tester un contrôle de conflit d'adresse. Membre application : ajouter un cas de test utile ou améliorer une réponse d'erreur. Membre données : renforcer un contrôle de réplication ou documenter une limite vérifiée. Membre validation : améliorer un script de mesure ou relier une preuve brute au rapport. Chacun doit aussi effectuer une petite contribution réelle hors de son rôle.

Contrôle des auteurs

Après les quatre contributions, exécuter git log --format=fuller --decorate --stat. Vérifier que les quatre noms et courriels attendus apparaissent et que chaque commit contient une modification identifiable. Sur GitHub ou GitLab, vérifier que les courriels sont liés et vérifiés sur les comptes correspondants.

Tableau à compléter

Membre 1 : nom _____ compte _____ branche _____ commit _____ test _____. Membre 2 : nom _____ compte _____ branche _____ commit _____ test _____. Membre 3 : nom _____ compte _____ branche _____ commit _____ test _____. Membre 4 : nom _____ compte _____ branche _____ commit _____ test _____.

Avant la remise

Vérifier git status, git log --oneline --decorate --all et git remote -v. Confirmer que .env, .runtime, .vagrant et les mots de passe ne figurent ni dans l'index ni dans l'historique. Exporter enfin une nouvelle archive depuis l'état validé et inscrire les identifiants réels dans le journal de groupe.